MÓDULO 6 DE 6 | AVANÇADO

Avançado, Boas Práticas e Publicação

Último módulo: elementos modernos do HTML, noções de JavaScript, performance, ferramentas e como colocar seu site no ar. Termina com um projeto final que junta tudo o que você aprendeu.

O que você vai aprender

- Elementos interativos: `details`, `dialog`, `template`
- Primeiros passos com JavaScript no HTML
- Performance e boas práticas de código
- Validar e depurar seu HTML
- Publicar o site e projeto final

1. Elementos interativos nativos

HTML

```html
<details>
  <summary>Qual o prazo de entrega?</summary>
  <p>De 3 a 5 dias  teis.</p>
</details>

<progress value="70" max="100">70%</progress>
<meter value="0.6">60%</meter>

<dialog id="aviso">
  <p>Mensagem importante.</p>
  <button id="fechar">Fechar</button>
</dialog>
```

`<details>` e `<summary>` criam um acordeão (perguntas frequentes) sem nenhum JavaScript. `<dialog>` cria janelas modais controladas por poucas linhas de script.

2. JavaScript no HTML

HTML

```html
<button id="abrir">Mostrar aviso</button>
<dialog id="aviso">
  <p>Ol !</p>
  <button id="fechar">Fechar</button>
</dialog>

<script src="js/app.js" defer></script>
```

js/app.js

```js
/* js/app.js */
const aviso  = document.getElementById("aviso");
const abrir  = document.getElementById("abrir");
const fechar = document.getElementById("fechar");

abrir.addEventListener("click", () => aviso.showModal());
fechar.addEventListener("click", () => aviso.close());
```

- O JavaScript **encontra** elementos (`getElementById`, `querySelector`), **escuta** eventos (`addEventListener`) e **altera** a página.
- `defer` no `<script>` faz o arquivo carregar sem travar a página e rodar depois que o HTML estiver pronto.
- Mantenha o JavaScript em arquivo externo, como o CSS.

Em português claro

Leia o script assim, linha a linha: (1) `getElementById("aviso")` procura na página a caixa com `id="aviso"` e guarda numa "gaveta" chamada `aviso`; (2) as outras duas linhas fazem o mesmo para os botões; (3) `addEventListener("click", ...)` significa "quando alguém clicar neste botão, execute esta ação"; (4) `aviso.showModal()` abre a janela e `aviso.close()` a fecha. O `=>` é só uma forma curta de escrever uma função.

Erro comum

Se o botão não faz nada, aperte F12 e abra a aba **Console**: ela mostra a mensagem de erro. Os motivos mais comuns são um `id` escrito diferente no HTML e no JavaScript e o caminho errado do arquivo no `<script src>`.

3. Performance

- Comprima imagens e use `width`/`height` para evitar saltos de layout.
- Use `loading="lazy"` em imagens abaixo da dobra.

- Carregue só o que usa: fontes, bibliotecas e scripts extras pesam.
- Minifique CSS e JS na hora de publicar.
- Meça com o Lighthouse (nas ferramentas do desenvolvedor do navegador).

4. Boas práticas e depuração

- Indente o código e use minúsculas em tags e atributos.
- Feche todas as tags e use aspas nos valores dos atributos.
- Nada de CSS de layout em atributos HTML antigos (`align`, `bgcolor`).
- Valide seu HTML no validador oficial do W3C (validator.w3.org).
- Aprenda a usar as **Ferramentas do Desenvolvedor** (tecla F12): aba Elements para inspecionar, Console para erros e Network para ver o que carrega.
- Use Git para versionar o projeto; é o seu histórico e seu botão de desfazer.

5. Publicando o site

Como seu site é feito só de HTML, CSS, JS e imagens (um site estático), publicá-lo é simples. As opções mais comuns:

- **GitHub Pages**, **Netlify**, **Vercel** ou **Cloudflare Pages**: gratuitos para sites estáticos e fáceis de usar.
- **Hospedagem tradicional**: você envia os arquivos por FTP ou painel.
- **Domínio próprio** (como seusite.com.br): registrado à parte e apontado para a hospedagem.

> **Dica**
> Antes de publicar: confira se `index.html` está na raiz, se os caminhos são relativos, se todas as páginas têm `title` e `description` e se testou no celular.

5.1 Passo a passo: publicando no GitHub Pages

O GitHub Pages é gratuito e ideal para sites feitos só com HTML, CSS e JavaScript.

1. Crie uma conta gratuita em github.com.
2. Crie um repositório novo e público, com um nome como `meu-site`.
3. Envie os arquivos do site (pelo botão de upload do próprio GitHub ou pelos comandos abaixo).
4. No repositório, abra **Settings**, depois **Pages**, escolha a branch `main` e a pasta raiz, e salve.
5. Aguarde alguns minutos: o endereço do site aparece na própria tela, no formato `https://seu-usuario.github.io/meu-site/`.

Terminal

```
$ cd meu-site
$ git init
$ git add .
$ git commit -m "Primeira vers o do site"
$ git branch -M main
$ git remote add origin https://github.com/SEU-USUARIO/meu-site.git
$ git push -u origin main
```

Dica

Os nomes dos menus do GitHub podem mudar com o tempo. Se não encontrar algum botão, procure na documentação oficial do GitHub Pages.

6. Projeto final

Monte um site de 4 páginas sobre um tema seu (portfólio, negócio, hobby, projeto pessoal):

- `index.html`, `sobre.html`, `servicos.html` (ou equivalente) e `contato.html` com formulário.
- Estrutura semântica completa, menu em todas as páginas e rodapé.
- CSS externo, layout com Flexbox e responsivo (testado de 360px a 1440px).
- Imagens com `alt`, meta tags de SEO e favicon.
- Um recurso interativo simples (acordeão com `details` ou modal com JavaScript).
- Publicado em um serviço gratuito, com o link enviado a um amigo para testar.

7. Próximos passos

Terminando este curso, você domina a base de qualquer site. Para continuar evoluindo, estes são os caminhos mais naturais:

- **CSS Grid e animações** para layouts e efeitos mais elaborados.
- **JavaScript** mais a fundo: variáveis, condições, laços, funções e manipulação do DOM.
- **Git e GitHub** para guardar o histórico do seu código e trabalhar em equipe.
- **Frameworks** como React ou Vue, depois que o JavaScript estiver firme.
- **Acessibilidade e performance** como hábito em todo projeto.

8. Erros comuns de iniciante

Erro comum
O site publicado aparece sem estilo. Geralmente é o caminho do CSS. Use caminhos relativos (`css/estilo.css`), não os do seu computador (`C:/Users/...`).

Erro comum
Página inicial dá erro 404. O arquivo da página inicial precisa se chamar exatamente `index.html` (minúsculas) e estar na raiz.

Erro comum
Script que não funciona porque carregou antes do HTML. Coloque `defer` no `<script>`.

Erro comum
Publicar sem testar no celular. Abra o endereço no seu telefone antes de divulgar.

9. Glossário rápido

Termo	Significado
JavaScript	Linguagem que dá comportamento e interação às páginas.
Evento	Algo que acontece na página, como um clique.
DOM	Representação da página que o JavaScript consegue ler e alterar.
Console	Aba das ferramentas do desenvolvedor que mostra erros e mensagens.
Git	Sistema que guarda o histórico de versões do seu código.
Repositório	Pasta de projeto controlada pelo Git, hospedada no GitHub.
Deploy / publicar	Colocar o site no ar para qualquer pessoa acessar.

Exercícios práticos

1. Crie uma página de perguntas frequentes usando `<details>`.
2. Escreva um script que mostra um `<dialog>` ao clicar em um botão.
3. Rode o Lighthouse no seu site e corrija pelo menos dois pontos apontados.
4. Valide todas as páginas no validador do W3C.
5. Publique o projeto final e teste o link no seu celular.

Resumo do módulo

- `details`, `dialog` e afins entregam interatividade sem esforço.

- JavaScript encontra elementos, escuta eventos e altera a página.
- Performance: imagens leves, scripts com `defer`, nada de excesso.
- Valide, depure com F12 e versione com Git.
- Sites estáticos são fáceis e baratos de publicar.